



デビルハンター
ヨミ

プログラム説明資料

北海道情報専門学校ゲームクリエイタ科 高橋英士

目次

- ①ダンジョンの自動生成プログラム
- ②各種スクリプトの一括管理
- ③敵AIについて

ダンジョンの自動生成プログラム

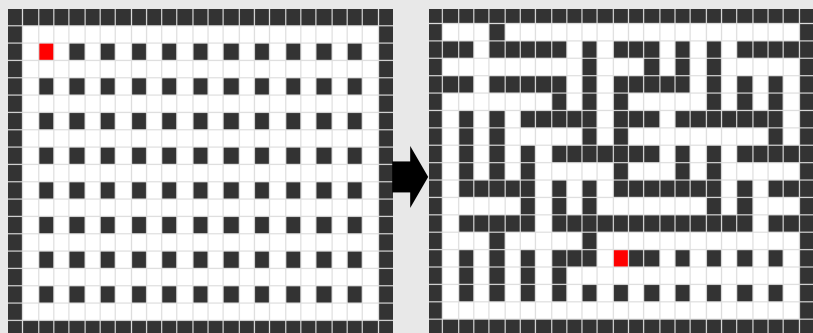
使用コード名：CreateDungeon.cs

このゲームでは繰り返しプレイの助長をするため、プレイする度にダンジョン構造を自動生成しています。



※全て同じアングルで撮影

自動生成の方法に関して、今回の制作では二次元配列を利用した「棒倒し法」を採用しました。



一定間隔に「柱」を配置し、それらを上下左右に倒すことで迷路を作成するロジック。

【ソースコード例】 ※CreateDungeon.csを参照

まずは等間隔に柱を配置した初期状態を作成。

(SerializeFieldで大きさは指定可能)

次に、配置した柱に対し
上下左右に倒すか、もしくは
「倒さない」
「柱を撤去」 そのいずれかを選択

宝箱等ギミックに関しても、
行き止まりのみに配置など工夫。

以上の要件で作成した
二次元配列を元に、シーン上に
ブロックやギミックを配置していく。

→ステージ生成完了

```
// Summary:
// マップデータを初期化するメソッド
// Returns:
// Returns: _width, _heightの分岐されたマップデータの二次元配列を返す/returns
// 呼び出し
private int[,] ResetMapData()
{
    int[,] map = new int[_width, _height];
    for (int y = 0; y < _height; y++)
    {
        for (int x = 0; x < _width; x++)
        {
            if (y == 0 || y == _height - 1)
            {
                map[x, y] = WALL_INDEX;
            }
            else if (x == 0 || x == _width - 1)
            {
                map[x, y] = WALL_INDEX;
            }
            else if (x % 2 == 0 && y % 2 == 0)
            {
                map[x, y] = WALL_INDEX;
            }
            else
            {
                map[x, y] = PATH_INDEX;
            }
        }
    }
    return map;
}
```

```
// Summary:
// 生成されたダンジョンの初期状態をシーン上に配置するメソッド
// Returns:
// Returns: 生成されたダンジョンの初期状態をシーン上に配置するメソッド
// 呼び出し
private void InitializeDungeon()
{
    Vector2 startPosition = new Vector2(0, 0);
    Vector2 endPosition = new Vector2(_width - 1, _height - 1);
    Vector2[] pathPositions = new Vector2[_width * _height];
    for (int i = 0; i < pathPositions.Length; i++)
    {
        pathPositions[i] = new Vector2(i % _width, i / _width);
    }
    for (int i = 0; i < pathPositions.Length; i++)
    {
        Vector2[] pathPositions = new Vector2[_width * _height];
        for (int j = 0; j < pathPositions.Length; j++)
        {
            pathPositions[j] = new Vector2(j % _width, j / _width);
        }
        Vector2[] pathPositions = new Vector2[_width * _height];
        for (int j = 0; j < pathPositions.Length; j++)
        {
            pathPositions[j] = new Vector2(j % _width, j / _width);
        }
    }
}
```

スクリプトの一括管理

使用コード名：UpdateManager.cs, PlayerManager.cs

このゲームでは、UpdateやFixedUpdateなど毎フレーム実行される部分に関して一括管理するため、まとめて一つのスクリプトで実行させています。

【ソースコード例】 ※UpdateManager.csを参照

このように、SerializeFieldでUpdater.csを継承させたスクリプトを配列内に格納し、それらをまとめて実行するようにしています。

```
Unity スクリプト10 個の参照
public class Updater : MonoBehaviour

3 個の参照
public virtual void UpdateMethod()

9 個の参照
public virtual void FixedUpdateMethod()
```

Updaterクラスの内容。
それぞれのメソッドでUpdateで処理する内容とFixedUpdateで処理する内容を選択できるように

```
<summary>
アップデート処理をまとめて実行する用のスクリプト
【制作者：高橋英士】
【制作日時：2025/9/24】
</summary>
Unity スクリプト 12 件のアセット参照 12 個の参照
public class UpdateManager : MonoBehaviour

// フィールド変数、及び定数など

[Header("【スクリプト取得】")]
[SerializeField, Tooltip("Updateメソッドで実行したいスクリプトを入れる場所")]
private Updater[] _updaters = default;

[SerializeField, Tooltip("FixedUpdateメソッドで実行したいスクリプトを入れる場所")]
private Updater[] _fixedUpdaters = default;

// メソッド

<summary>
毎フレーム実行されるメソッド (入力処理などに使う、TimeScaleの影響を受けない)
</summary>
Unity メッセージ10 個の参照
private void Update()

// updaters配列に格納されているUpdaterを継承したスクリプトのメソッドを実行
foreach (Updater updater in _updaters)
{
    updater.UpdateMethod();
}

<summary>
一定時間ごとに実行されるメソッド (入力処理以外の処理に使う、TimeScaleの影響を受ける)
</summary>
Unity メッセージ10 個の参照
private void FixedUpdate()

// fixedUpdaters配列に格納されているUpdaterを継承したスクリプトのメソッドを実行
foreach (Updater fixedUpdater in _fixedUpdaters)
{
    fixedUpdater.FixedUpdateMethod();
}
```

【ソースコード例】 ※PlayerManager.csを参照

継承元のメソッドをOverrideし、
その中に各種機能のUpdateで実行したい
内容をまとめて一カ所で実行させる。

```
// メソッド

Unity メッセージ10 個の参照
private void Start()

{
    _playerInputManager.ToStart();
    _bulletPool.ToStart();
    _inputShot.ToStart();
    _playerCheckObject.ToStart();
    _playerState.ToStart();
    _playerReLoad.ToStart();
    _cameraMove.ToStart();
    _playerRespawnManager.ToStart();
}

<summary>
Updateで実行されるメソッド
</summary>
2 個の参照
public override void UpdateMethod()

{
    _playerInputManager.UpdateInput();
    _playerMove.MoveInput(_playerInputManager);
    _cameraMove.InputCameraMove(_playerInputManager);
    _knifeAttack.InputAttack(_playerInputManager);
    _inputShot.InputShotBullet(_playerInputManager);
    _playerReLoad.InputReLoad(_playerInputManager);
    _playerCheckObject.CheckAndInput(_playerInputManager);
}

<summary>
FixedUpdateで実行されるメソッド
</summary>
2 個の参照
public override void FixedUpdateMethod()

{
    _playerDodge.DodgeUpdater();
    _knifeAttack.AttackCoolDown();
    _playerReLoad.ReLoad();
    _playerRespawn.RespawnCheck();
    _playerSceneManager.PlayAudioCoolDown();
}
```

```
// フィールド変数、及び定数など

// ---【変数】---//

[Header("【スクリプト取得】")]
[Header("【プレイヤー自身のスクリプト】")]
[SerializeField] private PlayerState _playerState = default;
[SerializeField] private PlayerRespawnManager _playerRespawnManager = default;
[SerializeField] private PlayerInputManager _playerInputManager = default;
[SerializeField] private PlayerMove _playerMove = default;
[SerializeField] private PlayerDodge _playerDodge = default;
[SerializeField] private CameraMove _cameraMove = default;
[SerializeField] private PlayerRespawnManager _playerRespawn = default;
[SerializeField] private InputShot _inputShot = default;
[SerializeField] private PlayerReLoad _playerReLoad = default;
[SerializeField] private KnifeAttack _knifeAttack = default;
[SerializeField] private PlayerCheckObject _playerCheckObject = default;
[SerializeField] private PlayerSceneManager _playerSceneManager = default;

[Header("【プレイヤー外のスクリプト】")]
[SerializeField] private BulletPool _bulletPool = default;
[SerializeField] private ManagerDirector _managerDirector = default;
```

上部のフィールド変数部分。
プレイヤーの各機能を取得しています。

よりスクリプトの管理がしやすく、どのタイミングで何が起こっているか理解しやすく

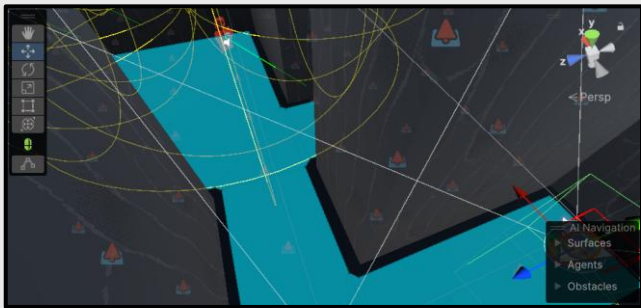
敵AIの制御プログラム

使用コード名：EnemyMove.cs, EnemyPlayerCheck.cs

より自然で違和感の無い敵AIの実装にも取り組みました。

- ・まず、敵AIの作成にあたって「NavMesh」を利用しています。

※EnemyMove.csを参照、壁を避けてプレイヤーを追って欲しいため実装しました。



図の「水色部分」に沿って、敵が移動するようになります。

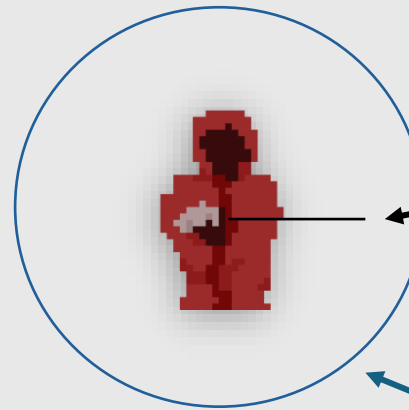
間に壁が挟まった場合でも自動で経路を計算し移動してくれます。

こちらを前提として、次のような実装をしました。

- ① ステートによってプレイヤー検知方法を変えることでより敵らしい動きを実現。
- ② 敵の徘徊システム。

①の実装 ※EnemyPlayerCheck.csを参照

検知判定例



- ・「RayCast」と「CheckCapsule」の二種類の判定を作成。
- ・敵には「通常状態」と「警戒状態」のステートを作成。

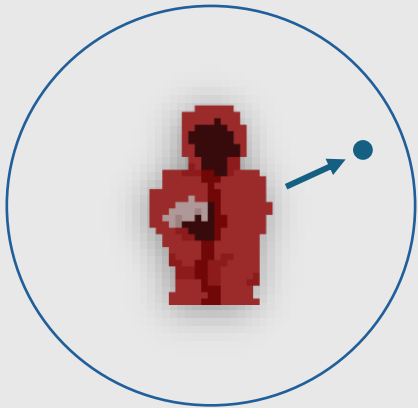
- ①通常状態ではRayCastでプレイヤーを判定することで、壁があった場合はプレイヤーを検知しないように。
- ②通常時にプレイヤーを検知するか、攻撃を受けると「警戒状態」に移行
- ③警戒状態ではCheckCapsuleで球状範囲内にプレイヤーがいる限り追いつける。
- ④球状範囲外にプレイヤーが出たら、通常状態に移行する。

検知方法を変える事で、より生きているような動きを実現。

敵AIの制御プログラム

②の実装 ※EnemyMove.csを参照

- ・プレイヤー未発見時、ウロウロと探索するような動きを以下のような動作を実装しました。



①未発見時、一定時間ごとに円形範囲内のNavMeshが割り当てられている地面のランダムな地点を取得。

②そこに向かって移動させる。

※徘徊させるタイミングの時間は、minとmaxを設定してランダムに計算

この動作の実装で、より生きた敵のような動きに。